

Implementation Approach and Design Rationale

Distributed Unary Inclusion Dependency Mining with Akka Typed

1. Overview

The implemented system realizes **real unary inclusion dependency (IND) discovery** using an actor-based architecture built with **Akka Typed**. The objective is not only to demonstrate distributed coordination, but to perform actual value-based inclusion checks across multiple relations from the TPCCH dataset.

Unlike the original prototype framework, which focused on execution flow, this implementation performs exhaustive unary IND detection based on real column data. The system combines distributed execution, streaming-based input handling, and deterministic termination while preserving the provided actor architecture.

2. System Architecture

The system consists of four primary actor types:

- **DependencyMiner** – central coordinator and task scheduler
- **DependencyWorker** – parallel execution units
- **InputReader** – streaming-based data ingestion
- **ResultCollector** – result aggregation and persistence

Each actor has a single responsibility and communicates exclusively through immutable messages. There is no shared mutable state, ensuring clear separation of concerns and fault isolation.

The architecture follows a master-worker pattern, where the miner coordinates work but delegates computational tasks to workers.

3. Data Collection and Preparation

For each input relation, an InputReader actor:

1. Reads the header to extract attribute names
2. Streams tuples in fixed-size batches
3. Forwards batches to the miner

The DependencyMiner aggregates column values by constructing a:

`Set<String>`

for every attribute across all relations.

This structure enables efficient inclusion testing using set containment. Streaming ensures scalable file processing while keeping I/O separated from computation logic.

4. Candidate Generation

After all input files are processed, the miner generates the complete search space of unary IND candidates.

For every pair of distinct attributes (A, B), where $A \neq B$, a task is created. This includes:

- Inter-relation comparisons
- Intra-relation comparisons

This exhaustive approach guarantees completeness for unary inclusion dependency discovery.

5. Distributed Inclusion Checking

Each DependencyWorker receives a task containing:

- Identifier of column A
- Identifier of column B
- Corresponding value sets

The worker evaluates the formal unary IND condition:

$$A \subseteq B \Leftrightarrow \text{valuesB.containsAll(valuesA)}$$

If the condition holds, the worker reports success to the miner.

To ensure safe and efficient serialization, workers transmit only primitive information (column identifiers and boolean result). The InclusionDependency object is reconstructed inside the miner before being forwarded to the result collector.

This design keeps communication lightweight and avoids complex object serialization issues.

6. Deterministic Task Tracking and Termination

The system does not rely on time-based stopping or artificial delays.

Instead, the miner maintains counters for:

- Total generated tasks
- Completed tasks

Termination is triggered only when:

finishedTasks == totalTasks

At that point:

1. A FinalizeMessage is sent to the result collector
2. Results are flushed to disk
3. The actor system shuts down via coordinated termination

This guarantees correctness and reproducibility.

7. Result Handling

The ResultCollector accumulates confirmed inclusion dependencies and writes them to result.txt during finalization. Separating result persistence from computation improves modularity and maintainability.

8. Design Strengths and Uniqueness

This implementation transforms the original framework into a functional unary IND miner while preserving its actor-based structure.

Key improvements include:

- Real value-based inclusion checking
- Removal of random dependency generation
- Elimination of time-based termination
- Exhaustive candidate evaluation
- Deterministic task completion tracking
- Fully distributed execution using dynamic worker registration

Although the system performs exhaustive inclusion checking without semantic pruning (which may produce additional valid but non-foreign-key INDs), it correctly implements the formal definition of unary inclusion dependency.

9. Conclusion

The solution demonstrates how inclusion dependency discovery can be structured as a distributed, message-driven computation model.

By separating input streaming, task coordination, parallel evaluation, and result persistence, the system achieves clarity, scalability, and correctness. The actor-based architecture allows natural

parallelism and provides a strong foundation for future extensions such as pruning strategies or n-ary dependency discovery.

The final system represents a complete and functional distributed unary inclusion dependency miner implemented within the Akka Typed framework.

